REPORT:TWITTER SENTIMENT ANALYSIS

***Abstract*— This report presents a comprehensive approach to sentiment classification on Twitter by using the SemEval 2017 Task 4 dataset. With the on-demand popularity for social media, it is now essential for researchers, corporations, and governments to understand public opinion. Sentiment analysis, however, has several difficulties since social media language is informal and dynamic. This report aims to address these issues by creating an effective analysis system that can consistently classify tweets as positive, negative, or neutral.The methodology includes an in-depth preprocessing stage to clean and standardize the data, maintaining sentiment-relevant components and eliminating noise. A variety of feature extraction techniques, including Bag of Words (BoW), TF-IDF, GloVe embeddings, and Word2Vec are explored, to capture the semantic depth of the language used on Twitter. Furthermore, performance of other machine learning models, including Decision Trees, Logistic Regression, and Support Vector Machines (SVM) are assessed, in categorizing tweet sentiments. The report focuses on innovative methods to preprocessing and feature extraction with the goal of improving the models' capacity to generalize across unknown datasets. The results show how well traditional machine learning models and modern NLP approaches work together to classify sentiment.**

Keywords - Sentiment classification, Twitter data, preprocessing, feature extraction, Bag of Words, TF-IDF, GloVe embeddings, Word2Vec, machine learning, Decision Trees, Logistic Regression, Support Vector Machines (SVM), deep learning, Long short-term memory (LSTM).

I. Introduction

In a world where digital communication holds major domination, sentiment analysis has become an essential technique for understanding and analyzing the immense amount of text data generated by users on social media posts, online reviews, and other online platforms. This report's main objective is sentiment classification on Twitter using the SemEval 2017 Task 4 dataset. This dataset is a component of the ongoing SemEval (Semantic Evaluation) series of contests focused on text semantic analysis. Task 4 focuses on categorizing tweet emotions into three groups: positive, negative, and neutral. Twitter, with its diverse user-generated content, presents an exciting space for sentiment analysis. Accurate sentiment categorization is, however, made more difficult by the platform's character restriction, the creative and informal use of language, such as slang, hashtags, and abbreviations, as well as the quick development of online vocabulary. The main goal of the project is to create an advanced sentiment classifier that fits precisely to the features of Twitter data. This means coming up with efficient preprocessing techniques to clean and normalize tweets, investigating different feature extraction techniques to extract the main sentiment communicated, and setting in place machine learning algorithms that can interpret tweets' complex sentiment. Given the dynamic nature of social media language, the secondary objective is to make sure the model is flexible and generalizable across various datasets, thereby increasing its applicability in real world scenarios. The findings contribute to the continuous development of more sophisticated and flexible sentiment analysis systems by providing insightful information on the complex aspects of sentiment analysis on social media.

II. The Datasets

This sentiment analysis on Twitter using datasets from SemEval 2017 Task 4 includes a variety of social media-specific variables, such as as user mentions, URLs, emoticons, hashtags, emojis, and different types of online slang as shown in Figure 2.They are composed of a varied collection of tweets that have been methodically classified as positive, negative, or neutral as shown in figure 1.

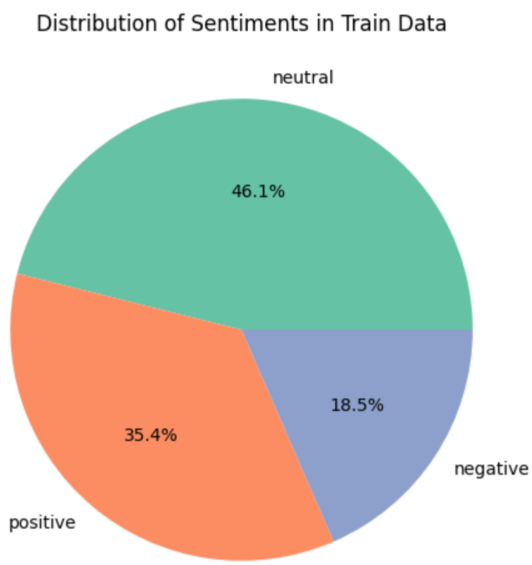

Fig. 1. Overview of the Dataset Structure and Characteristics

The collection comprises three different test datasets, a development dataset, and a training dataset. Each dataset offers a diverse selection of tweets that represent a wide range of real-world sentiment expression seen on the social media. The training dataset is used to develop and refine the classification models, enabling them to learn from examples of labelled sentiment. Throughout the iterative process of improvement, the development dataset is used as a key tool for fine-tuning the models and testing their functionality. To ensure that the performance evaluation appropriately represents the model's generalizability and adaptability across various sets of Twitter data, the three test datasets are crucial for the final assessment of the model's ability to precisely categorize sentiment in unseen data.

```
             tweet_id sentiment  \
0  262696992176304465   neutral
1  410734138242126311  positive
2  237615985571058688   neutral
3   90473590077188360   neutral
4  450236582392850660  negative

                                          tweet_text
0  @Oirisheye Hey you! I'm gonna be in Dublin in ...
1  Literally so excited I'm going to a Sam Smith ...
2  @WINDmobile Will there be an option to buy the...
3  Our Little Ms. Philippines. 👏👏👏#littleMsPhilip...
4  @AngryRaiderFan I know. This, TPP, expanded wa...
```

Fig. 2. Overview of the Dataset Structure and Characteristics

III. DATA PREPROCESSING

Data preprocessing, which is a critical step in sentiment analysis, involves transforming raw data into a cleaner, more standardized format suitable for analysis, where the goal is to accurately interpret and classify the sentiment of text data. During this sentiment analysis preprocessing stage, the following components are carefully addressed and handled:

- Lowercase Conversion: This removes case sensitivity by standardizing the content.
- Elimination of User Mentions: Removes @mentions as they often don't add emotions.
- Removal of URLs: Since hyperlinks have no impact on emotion, they are eliminated.
- Eliminating Non-Alphanumeric Characters: Removes special characters and punctuation to highlight textual content.
- Managing Character Repetition: Compresses character recurrences to standard spelling.
- Eliminates "RT" markings that indicate content that has been retweeted.
- Hashtag processing: takes off the '#' sign while keeping the hashtag's content.
- Numerical Data Removal: Removes numbers that don't usually relate to sentiments.
- Removal of Emojis and Emoticons: To standardize text processing, emojis and emoticons are eliminated.
- Whitespace Normalization: For uniformity, multiple white spaces into one.
- Trimming: Eliminates following and leading white spaces from text.
- Eliminating Text from Square Brackets: Removes any text that appears in square brackets.
- Tokenization: Divides the text into discrete words or "tokens" to allow for further analysis.

To improve the dataset, a customized list of stop words, containing certain non-sentiment words associated with time and online slang (such as days of the week, "today," "tomorrow," etc.), is included. Stop words are words that are cut out either before or after text is processed since they are typically frequent words with minimal or no meaning in respect to sentiment analysis. In this specific sentiment analysis task, stemming and lemmatization are not performed, in contrast to common practice in many NLP tasks. The decision to skip these phases was made strategically because Twitter data contains informal speech, slang, and words misspelled creatively to convey emotion. Lemmatization and stemming have the potential to misunderstand or oversimplify these phrases, which could weaken the sentiment being expressed and risk the sensitivity and accuracy of the sentiment analysis.Through preprocessing, the dataset is made more focused and manageable for the upcoming analysis as shown in Figure 3.

```
             tweet_id sentiment  \
0  163361196206957578   neutral
1  768006053969268950  negative
2  742616104384772304   neutral
3  102313285628711403   neutral
4  653274888624828198   neutral

                                          tweet_text  \
0  Candids: Heading to the Chateau Marmont in Wes...
1  @Dont__KAY_me omg same I was reading it in sch...
2    Watching MTV  Hits! The Wanted Chasing the sun!
3  Bing one-ups knowledge graph, hires Encyclopae...
4  On Thursday, concealed-carry gun license holde...

                                processed_tweet_text
0  candids heading chateau marmont west hollywood...
1                 omg reading school pssas sat crying
2                watching mtv hits wanted chasing sun
3  bing one ups knowledge graph hires encyclopaed...
```

Fig. 3. Overview of the preprocessed Dataset

IV. FEATURE EXTRACTION

In the exploration of text data derived from preprocessed tweets, four distinct feature extraction techniques were implemented: Bag of Words (BoW), TF-IDF, GloVe Embeddings, and Word2Vec Embeddings. Every technique takes a different approach to text data representation, contributing to deep exploration of feature extraction strategies.

A. Bag of Words (BoW)

"CountVectorizer" is used to construct the Bag of Words model, which converts tweets into a matrix of token counts. This approach is simple

yet effective, enabling models to identify the presence or absence of certain phrases in tweets **?**. The training data that has been converted into BoW features is represented by the variable “`X_train_bow`”; test and development data went through similar transformations.
The ability to measure the frequency of sentiment-laden phrases makes it more suitable for this task. It does, however, ignore word order and grammar, which might make it more difficult to grasp the context.

B. TF-IDF Vectorization

TF-IDF, implemented through TfidfVectorizer under the function "`apply_tfidf_feature_extraction`", refines the BoW approach by considering the overall document weightage of a word, helping to prioritize words that are unique to a document. By balancing the word frequency throughout the dataset, this technique reduces the bias towards terms that appear often in all tweets. The altered training and development data are represented by the variables “`X_train_tfidf`” and “`X_dev_tfidff`”, which improve the model’s capacity to concentrate on more appropriate characteristics for sentiment classification.
TF-IDF was chosen as it works very well for sentiment analysis on heterogeneous datasets such as Twitter, where it’s important to distinguish between **?**phrases that are common language and those that are unique to a certain sentiment. TF-IDF offers a sophisticated feature set for classification tasks by decreasing the weight of frequent words and highlighting terms that are more expressive of emotion.

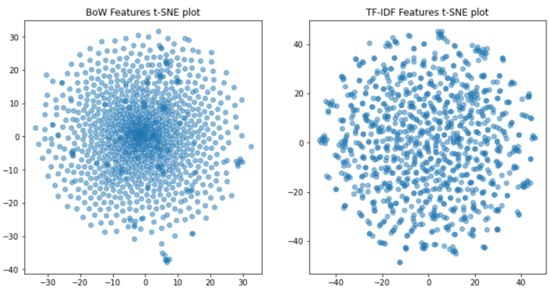

Fig. 4. The t-SNE visualization of BoW features and TF-IDF

In figure 4,The t-SNE visualization of BoW features shows a dense, central clustering, indicating that many tweets share common words, while TF-IDF features are more dispersed, suggesting a greater distinction between tweets. This dispersion reflects TF-IDF’s emphasis on unique words, providing potentially better feature separation for sentiment analysis.

C. GloVe Embeddings

Global Vectors for Word Representation, or GloVe Embeddings, loaded from a pre-trained model, offer a dense representation of words based on their co-occurrence statistics. The semantic similarity between words is captured by this technique. GloVe vectors are averaged and stored in `glove_train` and `glove_dev` variables for training and development data, respectively, using the "`tweet_to_glove_embedding`" function. This method provides a strong feature for sentiment analysis as it captures the fundamental semantic qualities of words. GloVe is especially well-suited for Twitter **?** data as it can capture the contextual clues necessary for understanding sentiment. It is skilled at managing the complexity of social media language because of its capacity to represent words in a multidimensional space depending on their context within a big corpus.

D. Word2Vec Embeddings

Another word embedding method called Word2Vec is built with the help of the Gensim package. It uses a dataset’s context to generate vector representations of words. Using the tokenized tweets, the function "`tweets_to_word2vec_embeddings`" trains a Word2Vec model to produce vectors that represent grammatical and semantic word associations. Variables `X_train_word2vec` and `X_dev_word2vec` represent the training and development datasets transformed into Word2Vec embeddings. This technique offers an array of characteristics for sentiment analysis and is good at capturing the nuances of language in tweets.
Word2Vec’s unique suitability for this task comes from its dynamic learning of word associations from the corpus (Twitter data). It captures the mood conveyed in original or unusual phrase use, adjusting effectively to the changing language of social media. The shapes of Extracted Features is shown in figure 5. Each technique brings a unique perspective to the analysis, from the simple frequency counts of BoW to the sophisticated semantic representations of GloVe and Word2Vec. Strategically, the analysis stands out by not just applying these methods in isolation but exploring their synergistic potential.

V. MODEL BUILDING

In the implementation of sentiment analysis classifiers, four diverse approaches were taken, each harnessing a different feature representation: Bag of Words (BoW), TF-IDF, GloVe embeddings, and Word2Vec embeddings. The clas-

```
Training BoW Features Shape: (45026, 55935)
Development BoW Features Shape: (2000, 55935)
Test Set 1 BoW Features Shape: (3531, 55935)
Test Set 2 BoW Features Shape: (1853, 55935)
Test Set 3 BoW Features Shape: (2379, 55935)
Training TF-IDF Features Shape: (45026, 10000)
Development TF-IDF Features Shape: (2000, 10000)
Test Set 1 TF-IDF Features Shape: (3531, 10000)
Test Set 2 TF-IDF Features Shape: (1853, 10000)
Test Set 3 TF-IDF Features Shape: (2379, 10000)
Training Word2Vec Embeddings Shape: (45026, 500)
Development Word2Vec Embeddings Shape: (2000, 500)
Test Set 1 Word2Vec Embeddings Shape: (3531, 500)
Test Set 2 Word2Vec Embeddings Shape: (1853, 500)
Test Set 3 Word2Vec Embeddings Shape: (2379, 500)
```

Fig. 5. The t-SNE visualization of BoW features and TF-IDF

sifiers chosen were Support Vector Machines (SVM)**?**, Logistic Regression, Decision Trees, and a Long Short-Term Memory (LSTM) network. Each classifier's suitability for the task was evaluated, considering the nature of the features it was paired with.

A. Support Vector Machine (SVM)

: SVM was used first as it is well-known for working well in high-dimensional domains. The linear kernel was selected because to its effectiveness in managing sparse data, a feature shared by TF-IDF and BoW representations. Using the SVC class from sklearn.svm, the interpretability and simplicity of the model were preserved by setting the kernel parameter to 'linear'.
Prior to training the model, a pickle file was used to integrate a check for previously trained models. SVM classifiers were trained on all four feature sets, and the models were stored for later use if no saved model could be discovered. This saves time and computing resources since it eliminates the need for retraining when a trained model is already in place.

B. Logistic regression

Logistic regression was chosen as the backup classifier because to its reliability and effectiveness in both binary and multi-class classification scenarios. An examination for pre-existing models was done, much as the SVM. For binary classification tasks, Logistic Regression benefits from using the 'liblinear' solver; however, since the sentiment analysis work involves several classes, it was essential to use a multinomial loss solver, such 'lbfgs' or 'newton-cg'. Pickle was then used to serialize the model for later usage. Because the approach is linear, it is easy to implement for both BoW and TF-IDF features, but less so for GloVe and Word2Vec unless those features are appropriately reduced or translated.

C. Decision Trees Classifier

Decision Trees were selected because of their interpretability and non-parametric characteristics, which imply that no data distribution is assumed. A DecisionTreeClassifier from sklearn.tree was trained after a model search was conducted. The BoW **?** and TF-IDF features can benefit from decision trees' ability to manage feature interactions. However, because of the potential complexity of decision boundaries resulting from the dense and continuous nature of GloVe and Word2Vec embeddings, their performance are not as good as others.

D. long short-term memory (LSTM)

One kind of recurrent neural network (RNN) that excels at learning from sequences is long short-term memory (LSTM)., where the context and order of words are essential for understanding sentiment.
The GloVe feature set is subjected to selective application of the LSTM model. The reason behind this choice is that GloVe embeddings give pre-trained word vectors with rich representations of word meanings, capturing the syntactic and semantic context of words. LSTMs can benefit greatly from such embeddings as they can take advantage of the depth of information that they have over sequences. The architecture of the LSTM model begins with the definition of an LSTMClassifier class, which inherits from PyTorch's nn.Module . An LSTM layer and a fully connected layer (self.fc) that translates the LSTM's hidden state to the output space are both included in the classifier. The probabilities of each sentiment class are obtained by applying a softmax layer to the fully connected layer's output, which corresponds to the output space.
Tensors are constructed using the sentiment labels and GloVe embeddings to train the LSTM. The data is batch loaded using a DataLoader, which offers a framework that makes LSTM model training more effective. Throughout the training phase, the model iterates over epochs, using forward propagation, backpropagation to update the weights, and cross-entropy to quantify loss (ideal for multiple-class classification applications). For best results, hyperparameters like batch size, learning rate, input dimension, and hidden layer size are adjusted. The dimensionality of GloVe embeddings is reflected in the input size, while the number of sentiment categories and model complexity are represented by the hidden and output sizes, respectively.
The trained LSTM model is stored for resilience in a pickle file. This eliminates the requirement for retraining and allows for simple reloading of the trained model for prediction tasks. To

guarantee consistent predictions, the model is placed in evaluation mode, which reduces several training-specific behaviors and layers, such dropout. The most likely sentiment class for each tweet is identified by running the data through the algorithm and using the output probabilities.

VI. Results and discussion

Based on the results obtained from training various classifiers with different feature sets:

- SVM achieved moderate performance across all feature sets, with TF-IDF yielding the highest accuracy of around 55%.
- Logistic Regression also demonstrated moderate performance, with the highest accuracy of approximately 58% achieved with Bag of Words (BoW).
- Decision Tree models yielded lower accuracies compared to SVM and Logistic Regression, with GloVe embeddings achieving the highest accuracy of around 35%.
- LSTM with GloVe embeddings achieved accuracy scores around 52%, indicating competitive performance compared to traditional machine learning models.

SVM and Logistic Regression classifiers performed better than Decision Trees, with SVM performing the best overall. Among the feature sets, TF-IDF with SVM and BoW with Logistic Regression showed the highest accuracies. The deep learning approach using LSTM with GloVe embeddings also showed promising results, demonstrating the potential of leveraging neural networks for sentiment analysis on Twitter data.

VII. Conclusion

GloVe embeddings seem to be generally effective in capturing semantic information for sentiment analysis, as demonstrated by their performance across different classifiers. SVM classifier shows good performance when combined with various feature extraction techniques, indicating its robustness in handling sentiment classification tasks on Twitter data.Logistic Regression also demonstrates competitive performance, particularly when applied to Bag of Words (BoW) features.

In conclusion, while the GloVe-SVM model emerges as the best performer, it's important to note that different combinations of feature extraction techniques and classifiers may suit different sentiment analysis tasks better. Further exploration and experimentation are warranted to determine the most suitable model for sentiment analysis on Twitter data. In comparison to GloVe and TF-IDF embeddings, Word2Vec embeddings coupled with SVM or Logistic Regression classifiers show lower performance in sentiment analysis on Twitter data. This indicates that for this specific task, GloVe and TF-IDF embeddings might be more suitable choices.

ref